

Sistema Distribuido para el Control y Administración de Laboratorios Informáticos en Entornos Académicos

Proyecto Final de Curso — Entrega 2: Arquitectura de Microservicios, Contrato API REST y Contenerización con Docker

Freddy Vladimir Farinango Guandinango — Arquitecto de Software
Isaías Abraham Urbina Romero — Líder de Desarrollo
Iván Andrés Villamarín Cuenca — Responsable de Calidad
Harold Nicolás Vinuesa Sánchez — Responsable de Documentación
Versión 2.0

Docente: Ing. Gleiston Cicerón Guerrero Ulloa

Fecha de entrega: 06 de junio de 2026 | Quevedo, Los Ríos — Ecuador

Resumen

El presente documento constituye la segunda entrega del Proyecto Final de Curso de la asignatura Aplicaciones Distribuidas (7mo nivel) de la Universidad Técnica Estatal de Quevedo. El proyecto tiene como objetivo optimizar la gestión de laboratorios informáticos mediante una arquitectura de microservicios que mejore la comunicación entre módulos, el acceso concurrente de usuarios y la administración de recursos académicos. Se documenta el diseño completo de la arquitectura de microservicios, compuesta por cinco servicios independientes (ms-usuarios, ms-reservas, ms-asistencia, ms-inventario y ms-reportes), coordinados a través de un API Gateway centralizado. Cada microservicio posee su propia base de datos PostgreSQL, garantizando el aislamiento de la persistencia de datos y eliminando el acoplamiento entre componentes. Se especifica el contrato completo de la API REST con métodos HTTP, rutas, estructuras de petición y respuesta en formato JSON para todos los endpoints del sistema. Adicionalmente, se define la estrategia de contenerización mediante Docker, incluyendo Dockerfiles individuales y un archivo docker-compose para la orquestación del entorno de desarrollo completo. El documento incluye diagramas C4 a tres niveles de abstracción, cinco Architecture Decision Records (ADR) y un plan de implementación semanal orientado a la Entrega 3.

Palabras clave: Microservicios; API Gateway; Docker; API REST; PostgreSQL; aplicaciones distribuidas; arquitectura de software; Spring Boot; sistemas académicos.

Abstract

This document constitutes the second deliverable of the Final Course Project for the Distributed Applications course (7th level) at the Universidad Técnica Estatal de Quevedo. The project aims to optimize the management of computer laboratories through a microservices architecture that improves inter-module communication, concurrent user access, and academic resource administration. The design adopts a microservices decomposition strategy aligned with Domain-Driven Design principles, where five independent services (ms-usuarios, ms-reservas, ms-asistencia, ms-inventario, and ms-reportes) are coordinated through a centralized API Gateway implemented with Spring Cloud Gateway. Each microservice maintains its own dedicated PostgreSQL database instance, applying the Database per Service pattern to ensure complete persistence isolation. The complete REST API contract is specified, including HTTP methods, routes, and JSON request/response structures for all system endpoints. C4 architectural diagrams at three levels of abstraction, five Architecture Decision Records (ADRs), and a weekly implementation plan leading to Deliverable 3 are included.

Keywords: microservices; API Gateway; Docker; REST API; PostgreSQL; distributed applications; software architecture; Spring Boot; academic systems.

I. REGISTRO DE CAMBIOS RESPECTO A LA ENTREGA 1

La siguiente tabla consolida las decisiones arquitectónicas que evolucionaron desde la Entrega 1, motivadas tanto por la retroalimentación recibida como por los requerimientos técnicos inherentes al diseño de un sistema verdaderamente distribuido. Cada cambio está justificado en términos de los principios de aplicaciones distribuidas estudiados en la asignatura.

Aspecto	Situación E1	Situación E2	Justificación
Arquitectura	Monolítica con módulos integrados en una sola aplicación Spring Boot	Microservicios independientes con API Gateway centralizado	El enfoque monolítico limita la escalabilidad horizontal y genera un único punto de fallo. Los microservicios permiten desplegar, escalar y mantener cada módulo de forma autónoma.
API Gateway	No definido. Cada módulo respondía directamente sin capa de enrutamiento	Spring Gateway como punto de entrada único para enrutamiento REST hacia los microservicios	Centralizar el acceso permite aplicar autenticación, balanceo de carga, logging y control de versiones desde un solo componente.
Base de datos	PostgreSQL unificada compartida por todos los módulos	Base de datos dedicada por microservicio (Database per Service) con PostgreSQL	La separación de esquemas garantiza aislamiento de datos, facilita migraciones independientes y elimina el acoplamiento a nivel de persistencia.
Contenerización	No contemplada en la Entrega 1	Docker con Dockerfile por servicio y docker-compose para orquestación	Docker garantiza reproducibilidad del entorno, elimina problemas de dependencias de plataforma y prepara la infraestructura para migración a Kubernetes.
Comunicación	Implícita mediante llamadas internas en la misma JVM	HTTP/REST entre microservicios independientes con contrato API documentado	REST sobre HTTP es el estándar en sistemas distribuidos modernos. Permite contratos claros, independencia de lenguaje y facilidad de prueba.
Metodología	Adaptación del PSP sin fases de integración de servicios	PSP adaptado con fases de diseño de contratos API, construcción de Dockerfiles y validación de integración entre contenedores	La ampliación de la metodología garantiza que cada entrega tenga artefactos trazables y coherentes con el estado real de implementación.

II. INTRODUCCIÓN

En la actualidad, los laboratorios informáticos representan un recurso importante dentro de las universidades porque permiten desarrollar prácticas académicas y actividades técnicas. Sin embargo, muchas instituciones todavía presentan problemas en la organización de horarios, control de reservas y seguimiento de equipos. Estas limitaciones generan conflictos en la asignación de laboratorios y dificultades para mantener información actualizada en tiempo real. Además, el crecimiento de usuarios y servicios digitales ha incrementado la necesidad de sistemas más eficientes y escalables dentro del entorno académico.

Actualmente existen diferentes sistemas orientados a la gestión de laboratorios informáticos, enfocados principalmente en reservas, inventario o monitoreo de equipos. Muchas de estas soluciones utilizan arquitecturas web tradicionales que centralizan el procesamiento y almacenamiento de la información. Aunque estos sistemas automatizan ciertos procesos administrativos, todavía presentan limitaciones relacionadas con el acceso concurrente y la comunicación entre módulos. Como resultado, aún existe una brecha en la integración de principios de aplicaciones distribuidas en plataformas universitarias de control de laboratorios.

A partir de esta problemática se plantean las siguientes preguntas de investigación: ¿la incorporación de principios de aplicaciones distribuidas mejora la comunicación entre módulos de un sistema de control de laboratorios informáticos?, y ¿el uso de una arquitectura distribuida permite optimizar el acceso concurrente y la disponibilidad de la información frente a un sistema centralizado? De manera complementaria, se establece la hipótesis de que la integración de servicios distribuidos permitirá mejorar la escalabilidad del sistema y reducir conflictos de acceso.

La contribución principal de este trabajo consiste en proponer la adaptación de un sistema web previamente desarrollado hacia una arquitectura orientada a aplicaciones distribuidas. La propuesta incorpora mecanismos de comunicación entre servicios, separación funcional de módulos y procesamiento concurrente para optimizar la gestión de reservas, asistencia, inventario y reportes técnicos.

II-A. Justificación del enfoque de microservicios

La arquitectura de microservicios propone descomponer una aplicación en un conjunto de servicios pequeños, independientes y desplegables de forma autónoma, cada uno ejecutando un proceso propio y comunicándose mediante mecanismos ligeros, habitualmente HTTP/REST [16]. Este enfoque responde directamente a las preguntas de investigación planteadas: si la incorporación de principios de aplicaciones distribuidas mejora la comunicación entre módulos, la respuesta arquitectónica es la definición de contratos explícitos de API entre servicios independientes [20]. Si una arquitectura distribuida optimiza el acceso concurrente, la respuesta técnica es la posibilidad de escalar horizontalmente cada servicio de forma independiente [21].

La necesidad de desacoplamiento funcional es imperativa en el contexto universitario: el módulo de reservas puede experimentar alta demanda al inicio del semestre, mientras que el módulo de reportes solo se consulta intensivamente en períodos de evaluación. Con microservicios, es posible asignar más instancias al servicio con mayor carga sin necesidad de replicar el sistema completo [22]. La disponibilidad se mejora porque el fallo de un microservicio no impide el funcionamiento del resto del ecosistema, siempre que se implementen patrones de resiliencia adecuados como circuit breakers y timeouts.

II-B. Alcance de la Entrega 2

Esta entrega se enfoca en el diseño completo de la arquitectura de microservicios, la especificación del contrato API REST y la estrategia de contenerización con Docker. No incluye evidencia experimental de implementación, la cual corresponde a la Entrega 3. El alcance cubre: diagramas C4 a tres niveles en código PlantUML, diseño detallado de los cinco microservicios, contrato API REST completo, Architecture Decision Records, Dockerfiles y

docker-compose, diagrama de despliegue y plan de implementación semanal.

III. REVISIÓN DEL ESTADO DEL ARTE

En los últimos años, las universidades han incrementado el uso de sistemas digitales para administrar laboratorios informáticos, reservas académicas y control de recursos tecnológicos. Diversos estudios muestran que la automatización de procesos permite reducir errores administrativos y mejorar la disponibilidad de información para estudiantes y docentes. Las primeras soluciones modernas estuvieron enfocadas principalmente en sistemas web centralizados con módulos básicos de horarios, inventario y registro de usuarios. Sin embargo, muchas de estas plataformas presentan limitaciones cuando múltiples usuarios acceden simultáneamente o cuando diferentes servicios deben compartir información en tiempo real [1], [2].

Un grupo importante de investigaciones se centra en sistemas de gestión académica basados en arquitecturas cliente-servidor tradicionales. Estos trabajos priorizan la facilidad de implementación y el almacenamiento centralizado de datos, permitiendo administrar reservas, reportes técnicos y control de asistencia desde una sola plataforma. Aunque este enfoque simplifica la administración inicial, también puede generar problemas de rendimiento y dependencia de un único punto de acceso. Como consecuencia, la escalabilidad y disponibilidad del sistema pueden verse afectadas cuando aumenta la cantidad de usuarios o procesos concurrentes [3], [4].

Otro conjunto de investigaciones aborda el uso de arquitecturas distribuidas y procesamiento concurrente para mejorar el intercambio de información entre servicios conectados en red. Estos estudios destacan que la separación funcional de módulos permite optimizar tiempos de respuesta y facilitar la ejecución simultánea de múltiples tareas. Además, el uso de componentes distribuidos mejora la tolerancia a fallos y la disponibilidad de los servicios dentro de sistemas complejos. A pesar de estos avances, gran parte de las propuestas analizadas están orientadas a sectores industriales, redes inteligentes o entornos de alto rendimiento computacional, dejando limitada su aplicación en plataformas universitarias de gestión de laboratorios [5], [6].

También se identifican investigaciones relacionadas con servicios distribuidos en entornos de edge computing y almacenamiento compartido. Estos trabajos proponen mecanismos de sincronización, intercambio de datos y distribución de cargas para optimizar el acceso concurrente a la información. Aunque los resultados muestran mejoras en rendimiento y comunicación entre procesos, las soluciones suelen enfocarse en infraestructura avanzada o servicios especializados. Por ello, todavía existe poca evidencia de propuestas distribuidas adaptadas específicamente a sistemas universitarios de control de laboratorios informáticos [7], [8], [9], [10].

En el contexto educativo, varios autores han desarrollado plataformas para reservas de laboratorios, monitoreo de equipos y gestión de inventario académico. La mayoría de estos sistemas integran funciones administrativas básicas y ofrecen interfaces web accesibles para docentes y estudiantes. No obstante, muchos trabajos se concentran únicamente en automatizar tareas específicas y no en mejorar la comunicación entre módulos mediante arquitecturas distribuidas. Esto evidencia una brecha relacionada con la falta de integración de servicios distribuidos en sistemas académicos orientados al control integral de laboratorios [11], [12].

A partir de la revisión realizada, se observa que los trabajos existentes aportan soluciones importantes en automatización, administración de recursos y procesamiento distribuido. Sin embargo, aún permanece sin resolver la integración de principios de aplicaciones distribuidas dentro de un sistema académico que combine reservas, asistencia, inventario y reportes técnicos en una sola arquitectura funcional. En consecuencia, existe la necesidad de proponer soluciones distribuidas adaptadas a la gestión integral de

laboratorios informáticos en instituciones educativas [13], [14], [15].

IV. TABLA COMPARATIVA DE TRABAJOS RELACIONADOS

Cuadro 1: Comparación de investigaciones relacionadas con sistemas distribuidos

Cita	Autores	Año	Método	Contexto	Métricas	Limitaciones
[1]	Yu et al.	2020	Sistema distribuido de almacenamiento	Cámaras y redes distribuidas	Privacidad y tolerancia a fallos	No orientado a educación
[2]	Syed et al.	2021	Acoplamiento distribuido en tiempo real	Infraestructuras geográficamente distribuidas	Sincronización y rendimiento	Aplicado a smart grids
[3]	Saleem et al.	2020	Framework de seguridad distribuida	Sistemas energía y control	Seguridad y disponibilidad	Enfoque energético
[4]	Peng et al.	2021	Control distribuido por eventos	Microrredes DC	Escalabilidad y estabilidad	Infraestructura especializada
[5]	Deng y Yang	2019	Control adaptativo tolerante fallos	Sistemas multiagente lineales	Estabilidad cooperativa	Contexto industrial
[6]	Sturz et al.	2020	Control predictivo distribuido	Sistemas lineales	Tiempo respuesta	No integra módulos académicos
[7]	Casimiro et al.	2020	Aprovisionamiento distribuido	Analítica distribuida	Eficiencia computacional	Orientado a empresas
[8]	Xu et al.	2020	Caché distribuida servicios	Mobile edge-clouds	Acceso concurrente	No aplicado educación
[9]	Mousavi et al.	2020	Aprendizaje colaborativo	Datos no etiquetados	Precisión clasificación	Procesamiento imágenes
[10]	Li et al.	2021	Cadenas Markov difusas	Sistemas inteligentes	Modelado y observación	No gestión académica
[11]	Wang et al.	2020	Gestión distribuida recursos	Hardware-in-the-loop	Rendimiento evaluación	Aplicación smart grids
[12]	Ridgley et al.	2020	Promedio distribuido privado	Sistemas distribuidos	Privacidad sincronización	No enfocado educación
[13]	Al-Saffar et al.	2020	Gestión con aprendizaje reforzado	Alta penetración PV	Mitigación sobrevoltaje	Contexto energético
[14]	Stafford y Treiblmaier	2020	Blockchain distribuido	Registros médicos	Compartición segura	No aplicado laboratorios
[15]	Batalla	2020	Transmisión distribuida video	Redes WiFi y 5G	Calidad transmisión	Enfoque IoT

V. SISTEMA PROPUESTO

La propuesta consiste en adaptar el sistema web de control de laboratorios hacia una arquitectura basada en aplicaciones distribuidas. Esto permitirá separar funciones como reservas, inventario, asistencia y reportes en módulos independientes que trabajen conectados entre sí. De esta manera, el sistema podrá atender varios usuarios al mismo tiempo y compartir información en tiempo real.

El módulo de reservas permitirá consultar horarios y solicitar laboratorios desde la plataforma web. El módulo de inventario ayudará a registrar equipos, mobiliario y reportes de fallas técnicas. Además, el sistema almacenará historiales de uso, mantenimientos y actividades realizadas en cada laboratorio.

La plataforma manejará distintos tipos de usuarios. Los estudiantes podrán consultar disponibilidad y realizar reservas, los docentes administrarán actividades académicas y el personal técnico controlará equipos y reportes. Esto permitirá que cada usuario tenga acceso únicamente a las funciones necesarias. La propuesta se diferencia de otros trabajos porque integra varias funciones académicas dentro de un mismo sistema distribuido.

VI. PREGUNTAS DE INVESTIGACIÓN E HIPÓTESIS

Las siguientes preguntas de investigación orientan el desarrollo del presente proyecto:

- ¿La incorporación de principios de aplicaciones distribuidas mejora la comunicación entre módulos de un sistema de control de laboratorios informáticos?
- ¿El uso de una arquitectura distribuida permite optimizar el acceso concurrente y la disponibilidad de la información frente a un sistema centralizado?

Hipótesis: la integración de servicios distribuidos permitirá mejorar la escalabilidad del sistema y reducir conflictos de acceso, facilitando además la administración académica y técnica de los laboratorios universitarios.

VII. OBJETIVOS

VII-A. Objetivo General

Optimizar la gestión de laboratorios informáticos mediante una arquitectura distribuida que mejore la comunicación entre módulos, el acceso simultáneo de usuarios y la administración de recursos académicos.

VII-B. Objetivos Específicos

- Integrar módulos distribuidos para reservas, asistencia, inventario y reportes técnicos.
- Implementar comunicación entre servicios para compartir información en tiempo real.
- Permitir el acceso concurrente de múltiples usuarios sin afectar el rendimiento del sistema.
- Separar funciones del sistema en módulos independientes para mejorar la escalabilidad y mantenimiento.
- Validar el funcionamiento coordinado de los módulos distribuidos dentro de la red universitaria.

VIII. JUSTIFICACIÓN

El proyecto se justifica por la necesidad de superar las limitaciones que presentan los sistemas centralizados en la gestión de laboratorios universitarios. La demanda creciente de acceso concurrente de estudiantes y docentes, combinada con la complejidad de administrar reservas, asistencia e inventario de forma coordinada, hace imprescindible una solución arquitectónica que garantice escalabilidad, disponibilidad y separación funcional de responsabilidades.

La arquitectura de microservicios, respaldada por la literatura académica reciente [16], [22], constituye la respuesta técnica más adecuada para este contexto. La adopción de contenedores Docker asegura la reproducibilidad del entorno y sienta las bases para una futura migración hacia plataformas de orquestación en la nube [19], [25]. El sistema propuesto no contempla la implementación de hardware especializado, inteligencia artificial o integración directa con plataformas institucionales externas.

IX. ARQUITECTURA GENERAL DEL SISTEMA

IX-A. Patrón Microservices Architecture

El sistema adopta el patrón Microservices Architecture, en el cual la aplicación se estructura como un conjunto de servicios de grano fino, cada uno con responsabilidad única y ciclo de vida independiente [16]. Esta decisión se fundamenta en los principios del Domain-Driven Design (DDD), donde cada microservicio encapsula un bounded context específico: el dominio de usuarios, el dominio de reservas, el dominio de asistencia, el dominio de inventario y el dominio de reportes [20]. Los microservicios se comunican exclusivamente a través de sus interfaces públicas (API REST), nunca mediante acceso directo a la base de datos de otro servicio.

IX-B. API Gateway

El API Gateway actúa como punto de entrada único al ecosistema de microservicios [23]. Implementado con Spring Cloud Gateway, cumple múltiples responsabilidades: enrutamiento de peticiones hacia el microservicio destino según la ruta URL, validación de tokens JWT emitidos por ms-usuarios antes de propagar la petición,

agregación de encabezados de seguridad, logging centralizado de todas las peticiones entrantes y gestión de CORS para el frontend web. El Gateway abstrae la complejidad interna del sistema: el cliente nunca necesita conocer la dirección o el puerto de cada microservicio individual.

IX-C. Comunicación REST y principios de diseño

Todos los microservicios exponen interfaces RESTful sobre HTTP/1.1 [24]. Los recursos se representan con sustantivos en plural en la URL (/api/reservas, /api/usuarios), los métodos HTTP definen la operación semántica (GET para consulta, POST para creación, PUT para actualización completa, PATCH para actualización parcial, DELETE para eliminación) y las respuestas utilizan códigos de estado HTTP estándar (200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Internal Server Error). Las representaciones de datos se codifican en JSON.

IX-D. Independencia y escalabilidad horizontal

La independencia de servicios se garantiza mediante tres principios: (1) cada microservicio tiene su propia base de datos y no puede acceder directamente a la base de datos de otro servicio; (2) cada microservicio tiene su propio repositorio Maven y su propio Dockerfile, lo que permite construirlo, desplegarlo y escalarlo de forma autónoma; (3) la comunicación entre servicios ocurre exclusivamente a través de llamadas HTTP a las APIs públicas [22]. La escalabilidad horizontal se logra desplegando múltiples instancias del mismo microservicio detrás del API Gateway [21].

X. ARQUITECTURA DE MICROSERVICIOS

Cada microservicio del sistema sigue un diseño uniforme basado en los principios de Spring Boot 3.x con Java 21 [16], utilizando las siguientes dependencias comunes:

- spring-boot-starter-web (servidor Tomcat embebido)
- spring-boot-starter-data-jpa (acceso a datos con Hibernate)
- postgresql (driver JDBC)
- spring-boot-starter-security y jwt (seguridad y tokens JWT en ms-usuarios)
- spring-boot-starter-validation (validación de entidades de entrada)

Todos los microservicios implementan el patrón Repository de Spring Data JPA, exponiendo interfaces que extienden JpaRepository<Entity, Long>. La capa de servicio encapsula la lógica de negocio e invoca los repositorios; los controladores REST únicamente gestionan la traducción entre HTTP y los objetos de dominio, manteniendo el principio de responsabilidad única [23].

La siguiente tabla resume los cinco microservicios del sistema:

Microservicio	Responsabilidad	Datos gestionados	Endpoints principales
ms-usuarios	Gestión de identidades, roles y autenticación JWT	usuarios, roles, tokens, sesiones	POST /registro, POST /login, GET /{id}, PUT /{id}, DELETE /{id}
ms-reservas	Control del ciclo de vida de las reservas: solicitud, aprobación, rechazo y cancelación	reservas, horarios, estado, laboratorio asignado	POST /reservas, GET /reservas, GET /{id}, PUT /{id}, DELETE /{id}
ms-asistencia	Registro y consulta de asistencia vinculado a reservas activas	registros de asistencia, fecha/hora, usuario, laboratorio	POST /asistencia, GET /asistencia, GET /{id}, GET /reporte
ms-inventario	Administración del inventario de equipos, mobiliario y recursos tecnológicos	equipos, categorías, estado, laboratorio asignado	POST /inventario, GET /inventario, GET /{id}, GET /laboratorio/{id}
ms-reportes	Generación de reportes técnicos, estadísticos y de uso	reportes generados, tipo, periodo,	POST /reportes, GET /reportes, GET /uso, GET /asistencia, GET /inventario

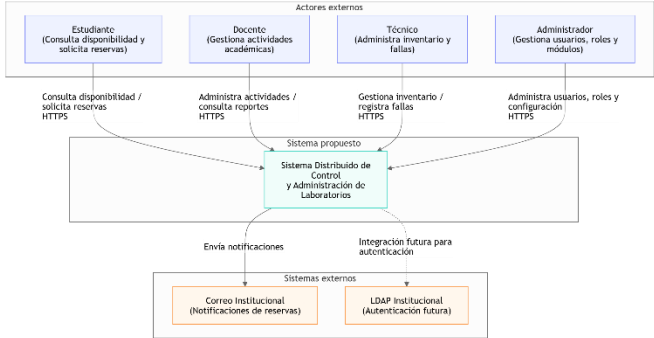
		usuario solicitante	
--	--	---------------------	--

XI. DIAGRAMAS C4

El modelo C4 (Context, Container, Component, Code) proporciona un conjunto de diagramas jerarquizados que describen la arquitectura de software a diferentes niveles de abstracción [17]. Esta sección presenta los tres primeros niveles del modelo aplicados al Sistema Distribuido de Control de Laboratorios, representados en imágenes a continuación.

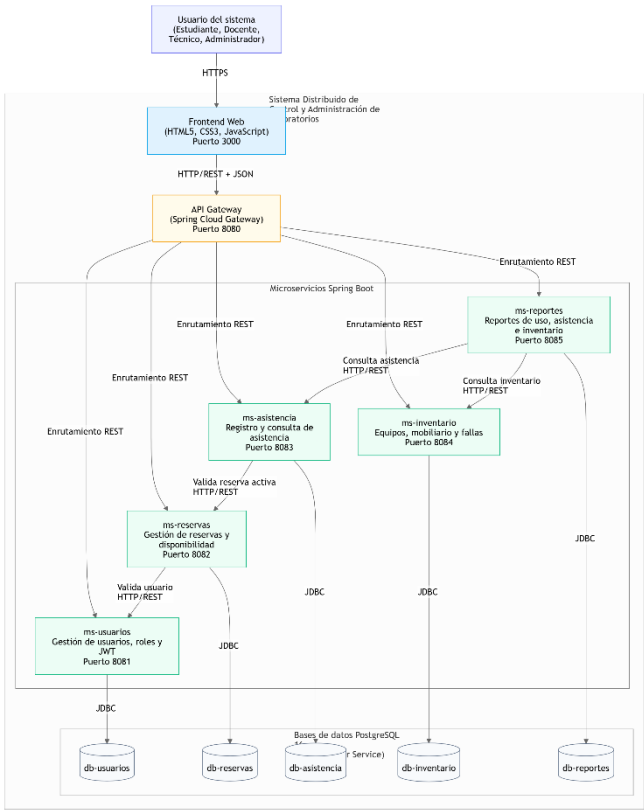
XI-A. Diagrama de Contexto (Nivel 1)

El diagrama de contexto muestra el sistema en su entorno más amplio, identificando los actores externos que interactúan con él y las relaciones de alto nivel. Los cuatro actores externos son: Estudiante (consulta disponibilidad y realiza reservas), Docente (administra actividades académicas y consulta reportes), Técnico (gestiona inventario de equipos y reportes de fallas) y Administrador (gestión global de usuarios, roles y módulos).



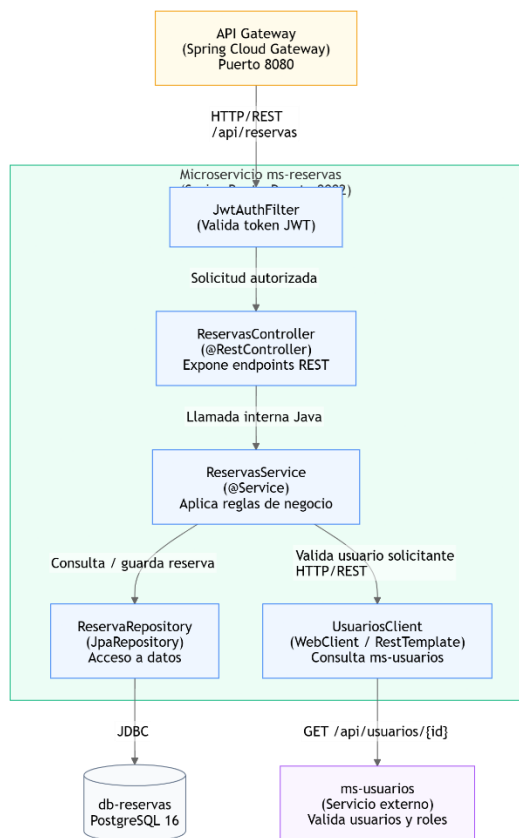
XI-B. Diagrama de Contenedores (Nivel 2)

El diagrama de contenedores desglosa el sistema en sus unidades desplegables principales, mostrando las tecnologías utilizadas y los protocolos de comunicación entre ellas. Cada contenedor tiene responsabilidad única y se comunica exclusivamente a través de interfaces explícitas.



XI-C. Diagrama de Componentes (Nivel 3) — ms-reservas

El diagrama de componentes detalla la estructura interna del microservicio ms-reservas, el componente de mayor centralidad funcional del sistema. La arquitectura interna sigue el patrón en capas de Spring Boot: controlador, servicio, repositorio y cliente REST externo.



XII. CONTRATO API REST

La siguiente especificación define el contrato completo de la API REST del sistema [24]. Todos los endpoints requieren el encabezado Authorization: Bearer <token_jwt> excepto POST /api/usuarios/registro y POST /api/usuarios/login. Las respuestas de error siguen el formato estándar: { "timestamp": "...", "status": 400, "error": "Bad Request", "message": "...", "path": "/api/..." }.

XII-A. ms-usuarios

Método	Endpoint	Descripción
POST	/api/usuarios/registro	Registrar usuario
POST	/api/usuarios/login	Iniciar sesión
GET	/api/usuarios/{id}	Consultar usuario
PUT	/api/usuarios/{id}	Actualizar usuario
DELETE	/api/usuarios/{id}	Desactivar usuario
GET	/api/usuarios	Listar usuarios

POST /api/usuarios/registro

Request:

```
{
  "nombre": "Juan Pérez",
  "email": "juan@uteq.edu.ec",
  "password": "SecurePass123",
  "rol": "ESTUDIANTE"
}
```

Response (201 Created):

```
{
```

```
  "id": 1,
  "nombre": "Juan Pérez",
  "email": "juan@uteq.edu.ec",
  "rol": "ESTUDIANTE",
  "activo": true,
  "creado": "2026-06-06T10:00:00"
}
```

POST /api/usuarios/login

Request:

```
{
  "email": "juan@uteq.edu.ec",
  "password": "SecurePass123"
}
```

Response (200 OK):

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "tipo": "Bearer",
  "expiracion": "2026-06-07T10:00:00",
  "rol": "ESTUDIANTE"
}
```

GET /api/usuarios/{id}

Response (200 OK):

```
{
  "id": 1,
  "nombre": "Juan Pérez",
  "email": "juan@uteq.edu.ec",
  "rol": "ESTUDIANTE",
  "activo": true
}
```

PUT /api/usuarios/{id}

Request:

```
{
  "nombre": "Juan Pérez Actualizado",
  "email": "juan2@uteq.edu.ec"
}
```

Response (200 OK):

```
{
  "id": 1,
  "nombre": "Juan Pérez Actualizado",
  "email": "juan2@uteq.edu.ec",
  "rol": "ESTUDIANTE",
  "activo": true
}
```

DELETE /api/usuarios/{id}

Response (200 OK):

```
{
  "mensaje": "Usuario con id 1 desactivado correctamente."
}
```

GET /api/usuarios

Response (200 OK):

```
[
  {
    "id": 1,
    "nombre": "Juan Pérez",
    "email": "juan@uteq.edu.ec",
    "rol": "ESTUDIANTE"
  }
]
```

XII-B. ms-reservas

Método	Endpoint	Descripción
POST	/api/reservas	Crear nueva reserva
GET	/api/reservas	Listar reservas
PUT	/api/reservas/{id}	Actualizar estado de reserva
DELETE	/api/reservas/{id}	Cancelar reserva
GET	/api/reservas/disponibilidad	Consultar disponibilidad de horarios

POST /api/reservas

Request:

```
{
  "usuarioId": 1,
  "laboratorioId": 3,
  "fechaInicio": "2026-06-10T08:00:00",
  "fechaFin": "2026-06-10T10:00:00",
  "motivo": "Práctica de Redes"
}
```

Response (201 Created):

```
{
  "id": 5,
  "usuarioId": 1,
  "laboratorioId": 3,
  "estado": "PENDIENTE",
  "fechaInicio": "2026-06-10T08:00:00",
  "fechaFin": "2026-06-10T10:00:00"
}
```

GET /api/reservas

Response (200 OK):

```
[
  {
    "id": 5,
    "estado": "APROBADA",
    "laboratorioId": 3
  }
]
```

PUT /api/reservas/{id}

Request:

```
{
  "estado": "APROBADA",
  "observacion": "Aprobado por coordinación"
}
```

Response (200 OK):

```
{
  "id": 5,
  "estado": "APROBADA",
  "observacion": "Aprobado por coordinación"
}
```

DELETE /api/reservas/{id}

Response (200 OK):

```
{
  "mensaje": "Reserva con id 5 cancelada correctamente."
}
```

GET /api/reservas/disponibilidad?laboratorioId=3&fecha=2026-06-10

Response (200 OK):

```
{
  "laboratorioId": 3,
  "fecha": "2026-06-10",
  "horasDisponibles": [
    "06:00",
    "08:00",
    "14:00"
  ]
}
```

XII-C. ms-asistencia

Método	Endpoint	Descripción
POST	/api/asistencia	Registrar asistencia de usuario
GET	/api/asistencia/reporte	Consultar reporte de asistencia
PUT	/api/asistencia/{id}	Corregir registro de asistencia (ADMINISTRADOR)

POST /api/asistencia

Request:

```
{
  "usuarioId": 1,
  "reservaId": 5,
  "tipo": "ENTRADA",
  "timestamp": "2026-06-10T08:05:00"
}
```

Response (201 Created):

```
{
  "id": 12,
  "usuarioId": 1,
  "reservaId": 5,
  "tipo": "ENTRADA",
  "timestamp": "2026-06-10T08:05:00",
  "validado": true
}
```

GET /api/asistencia/reporte?usuarioId=1&desde=2026-06-01&hasta=2026-06-30

Response (200 OK):

```
{
  "usuarioId": 1,
  "totalAsistencias": 8,
  "porcentaje": 88.9,
  "detalle": [
    ...
  ]
}
```

PUT /api/asistencia/{id}

Request:

```
{
  "tipo": "SALIDA",
  "observacion": "Corrección manual"
}
```

Response (200 OK):

```
{
  "id": 12,
  "tipo": "SALIDA",
  "observacion": "Corrección manual",
  "modificado": true
}
```

XII-D. ms-inventario

Método	Endpoint	Descripción
POST	/api/inventario	Registrar nuevo equipo
PUT	/api/inventario/{id}	Actualizar estado de equipo
GET	/api/inventario/laboratorio/{id}	Consultar inventario de un laboratorio

POST /api/inventario

Request:

```
{
  "nombre": "PC-Lab3-05",
  "categoria": "COMPUTADORA",
  "laboratorioId": 3,
  "estado": "OPERATIVO",
  "descripcion": "Intel Core i7, 16GB RAM, 512GB SSD"
}
```

Response (201 Created):

```
{
  "id": 45,
  "nombre": "PC-Lab3-05",
  "categoria": "COMPUTADORA",
  "laboratorioId": 3,
  "estado": "OPERATIVO"
}
```

PUT /api/inventario/{id}

Request:

```
{
  "estado": "EN_REPARACION",
  "observacion": "Falla en fuente de poder"
}
```

Response (200 OK):

```
{
  "id": 45,
  "estado": "EN_REPARACION",
  "observacion": "Falla en fuente de poder"
}
```

GET /api/inventario/laboratorio/{id}

Response (200 OK):

```
{
  "laboratorioId": 3,
  "totalEquipos": 30,
}
```



```

"operativos": 27,
"enReparacion": 2,
"deBaja": 1,
"equipos": [
  ...
]
}

```

XII-E. ms-reportes

Método	Endpoint	Descripción
POST	/api/reportes	Generar nuevo reporte
GET	/api/reportes/uso	Reporte de uso del laboratorio
GET	/api/reportes/asistencia	Reporte de asistencia
GET	/api/reportes/inventario	Estado del inventario

POST /api/reportes Generar nuevo reporte

Request body:

```

{
  "tipo": "USO_LABORATORIO",
  "laboratorioId": 3,
  "fechaInicio": "2026-06-01",
  "fechaFin": "2026-06-30",
  "solicitanteId": 2
}

```

Response 201 Created:

```

{
  "id": 9,
  "tipo": "USO_LABORATORIO",
  "estado": "GENERADO",
  "urlDescarga": "/api/reportes/9/descargar",
  "generadoEn": "2026-06-06T..."
}

```

GET /api/reportes/uso

Query:

?laboratorioId=3&desde=2026-06-01&hasta=2026-06-30

Response:

```

{
  "laboratorioId": 3,
  "totalReservas": 42,
  "horasUsadas": 84,
  "ocupacionPromedio": 70.5,
  "usuariosMasActivos": [ ... ]
}

```

```

}

```

GET /api/reportes/asistencia

Query:

?laboratorioId=3&periodo=2026-06

Response:

```

{
  "laboratorioId": 3,
  "periodo": "2026-06",
  "totalRegistros": 180,
  "promedioAsistencia": 85.2,
  "detallePorFecha": [ ... ]
}

```

GET /api/reportes/inventario — Estado del inventario

Query:

?laboratorioId=3

Response:

```

{
  "laboratorioId": 3,
  "totalEquipos": 30,
  "operativos": 27,
  "enReparacion": 2,
  "porcentajeOperatividad": 90.0
}

```

XIII. ARCHITECTURE DECISION RECORDS (ADR)

Los Architecture Decision Records documentan las decisiones arquitectónicas significativas tomadas durante el diseño del sistema, incluyendo el contexto que motivó la decisión, la decisión adoptada y las consecuencias previstas [18]. Cada ADR es un artefacto inmutable que preserva la trazabilidad de las decisiones a lo largo del ciclo de vida del proyecto.

ADR-001. Adopción de Arquitectura de Microservicios

Contexto: El sistema debe gestionar cinco dominios funcionales distintos con requisitos de disponibilidad y escalabilidad diferentes. Una arquitectura monolítica obligaría a desplegar toda la aplicación cada vez que un módulo cambia, y un fallo en un módulo podría derribar el sistema completo. El equipo de desarrollo está organizado por áreas funcionales, lo que favorece la autonomía de cada grupo [16].

Decisión: Se adopta la arquitectura de microservicios, en la que cada dominio funcional se implementa como un servicio independiente con su propio proceso de ejecución, su propia base de datos y su propio ciclo de despliegue. Los servicios se

comunican exclusivamente mediante APIs REST sobre HTTP [20].

Consecuencias positivas: escalabilidad independiente por servicio; fallos aislados; despliegues autónomos; equipos desacoplados [21].

Consecuencias negativas: incremento de la complejidad operativa; necesidad de un API Gateway; latencia de red en comunicación entre servicios; mayor esfuerzo de pruebas de integración.

ADR-002. Implementación de API Gateway con Spring Cloud Gateway

Contexto: Con cinco microservicios independientes, el frontend web necesitaría conocer la dirección y puerto de cada servicio. Esto generaría fuerte acoplamiento cliente-servidor, dificultaría la aplicación uniforme de autenticación y haría inmanejable la gestión de CORS, logging y versionado [23].

Decisión: Se implementa un API Gateway utilizando Spring Cloud Gateway. Centraliza el enrutamiento de peticiones según el prefijo de la URL, aplica filtros de autenticación JWT globales y gestiona CORS para el frontend.

Consecuencias positivas: el cliente trabaja con una única URL base; autenticación centralizada; punto único de monitoreo; facilidad de actualización de rutas internas sin cambiar el frontend.

Consecuencias negativas: el Gateway es un nuevo punto potencial de fallo, mitigado con redundancia; añade una capa de latencia mínima en red local.

ADR-003. Base de Datos Dedicada por Microservicio (Database per Service)

Contexto: Si todos los microservicios comparten una sola base de datos PostgreSQL, cualquier cambio en el esquema de un servicio puede romper los demás. El acceso directo de múltiples servicios a las mismas tablas genera un acoplamiento oculto a nivel de datos que contradice el principio de autonomía de microservicios [22].

Decisión: Se aplica el patrón Database per Service: cada microservicio tiene su propia instancia de PostgreSQL desplegada en un contenedor Docker separado con su propio esquema. Ningún microservicio puede acceder directamente a la base de datos de otro; toda consulta debe realizarse a través de la API REST del servicio correspondiente.

Consecuencias positivas: aislamiento completo de persistencia; migraciones independientes sin afectar otros servicios; posibilidad futura de migrar a diferentes motores de base de datos por servicio [16].

Consecuencias negativas: no hay transacciones distribuidas ACID entre servicios; la consistencia entre dominios es eventual; mayor número de contenedores Docker a gestionar.

ADR-004. Contenerización con Docker y Orquestación con docker-compose

Contexto: El equipo de desarrollo trabaja en distintos equipos con Windows 10/11. Las diferencias de entorno generan el problema clásico de 'funciona en mi máquina'. Además, el sistema necesita desplegar múltiples servicios coordinados con redes y volúmenes compartidos [19].

Decisión: Cada microservicio y cada base de datos se empaqueta en un contenedor Docker mediante un Dockerfile específico. Se utiliza docker-compose para definir y levantar el ecosistema completo con un único comando: docker-compose up. Se definen redes Docker internas y volúmenes persistentes para los datos de PostgreSQL.

Consecuencias positivas: reproducibilidad total del entorno; despliegue en un único comando; aislamiento de procesos; base para futura migración a Kubernetes [25].

Consecuencias negativas: curva de aprendizaje de Docker para el equipo; sobrecarga de recursos en equipos con poca RAM (mitigado con perfil de desarrollo ligero).

ADR-005. Comunicación Síncrona mediante HTTP/REST entre Microservicios

Contexto: Los microservicios necesitan intercambiar datos: ms-reservas debe validar usuarios en ms-usuarios; ms-asistencia debe verificar reservas en ms-reservas; ms-reportes debe agregar datos de ms-asistencia y ms-inventario. Se evaluaron tres opciones: REST síncrono, mensajería asíncrona (RabbitMQ/Kafka) y gRPC [24].

Decisión: Se adopta comunicación síncrona HTTP/REST para todas las interacciones entre microservicios en esta etapa del proyecto. Se utilizará RestTemplate o Spring WebClient para las llamadas entre servicios. La mensajería asíncrona queda como mejora futura.

Consecuencias positivas: simplicidad de implementación y depuración; contratos claros con JSON; compatible con las competencias del equipo; trazabilidad directa en logs [21].

Consecuencias negativas: acoplamiento temporal (el servicio destino debe estar disponible); riesgo de latencia acumulada en llamadas encadenadas; sin tolerancia a fallos inherente (requiere circuit breakers en iteraciones futuras).

XIV. ESTRATEGIA DE CONTENERIZACIÓN CON DOCKER

XIV-A. Fundamentos de Docker en el contexto del proyecto

Docker es una plataforma de contenerización que permite empaquetar una aplicación junto con todas sus dependencias en una unidad estandarizada llamada contenedor [19]. A diferencia de las máquinas virtuales, los contenedores comparten el kernel del sistema operativo huésped, lo que los hace significativamente más ligeros en términos de recursos. En el contexto de este proyecto, Docker garantiza que el entorno de ejecución de cada microservicio sea idéntico en todos los equipos del equipo de desarrollo, independientemente del sistema operativo del desarrollador [25].

XIV-B. Dockerfile para microservicios Spring Boot

Cada microservicio tiene un Dockerfile ubicado en la raíz de su módulo Maven. Se utiliza el patrón multi-stage build para reducir el tamaño de la imagen final: la primera etapa compila el código con Maven y la segunda etapa copia únicamente el JAR resultante a una imagen JRE mínima.

```
# Etapa 1: compilación
FROM eclipse-temurin:21-jdk-alpine AS builder
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

# Etapa 2: imagen de producción
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8082
ENTRYPOINT ["java", "-jar", "app.jar"]
```

La instrucción EXPOSE documenta el puerto que el servicio expone dentro de la red Docker. Los puertos por servicio son: 8080 (Gateway), 8081 (ms-usuarios), 8082 (ms-reservas), 8083 (ms-asistencia), 8084 (ms-inventario), 8085 (ms-reportes).

XIV-C. docker-compose: orquestación del ecosistema completo

El archivo docker-compose.yml define todos los contenedores del sistema, sus dependencias de inicio, la red interna compartida y los volúmenes para persistencia de datos de PostgreSQL.

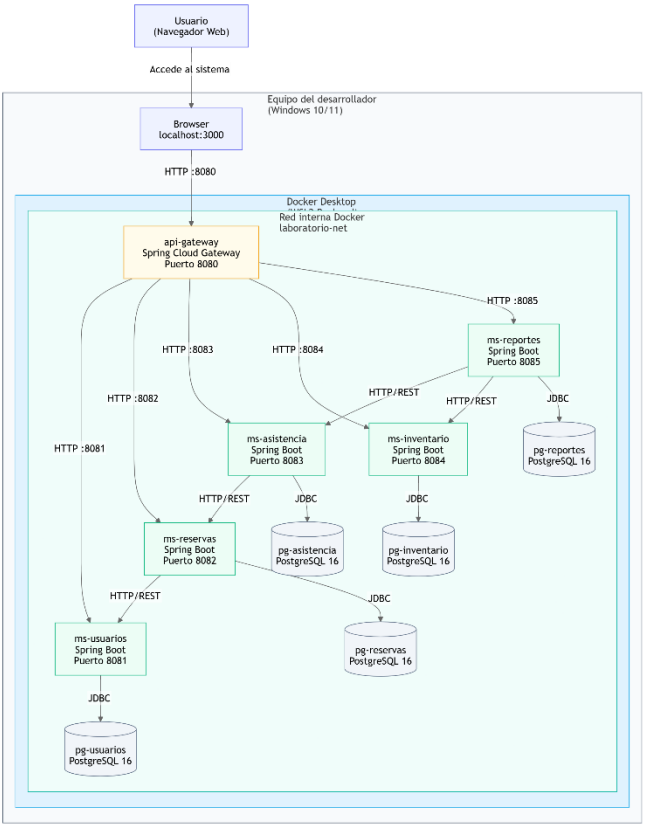
```
version: '3.9'
services:
  pg-usuarios:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: db_usuarios
      POSTGRES_USER: user_lab
      POSTGRES_PASSWORD: lab_pass_2026
    volumes:
      - pg-usuarios-data:/var/lib/postgresql/data
    networks: [laboratorio-net]
  pg-reservas:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: db_reservas
      POSTGRES_USER: user_lab
      POSTGRES_PASSWORD: lab_pass_2026
    volumes:
      - pg-reservas-data:/var/lib/postgresql/data
    networks: [laboratorio-net]
  # (pg-asistencia, pg-inventario, pg-reportes:
  # misma estructura)
  ms-usuarios:
    build: ./ms-usuarios
    ports: ['8081:8081']
    depends_on: [pg-usuarios]
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://
        pg-usuarios:5432/db_usuarios
      SPRING_DATASOURCE_USERNAME: user_lab
      SPRING_DATASOURCE_PASSWORD: lab_pass_2026
      JWT_SECRET: clave_secreta_jwt_uteq_2026
    networks: [laboratorio-net]
  ms-reservas:
    build: ./ms-reservas
    ports: ['8082:8082']
    depends_on: [pg-reservas, ms-usuarios]
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://
        pg-reservas:5432/db_reservas
      MS_USUARIOS_URL: http://ms-usuarios:8081
    networks: [laboratorio-net]
  # (ms-asistencia, ms-inventario, ms-reportes:
  # misma estructura)
  api-gateway:
    build: ./api-gateway
    ports: ['8080:8080']
    depends_on:
      - ms-usuarios
      - ms-reservas
      - ms-asistencia
      - ms-inventario
      - ms-reportes
    environment:
      MS_USUARIOS_URL: http://ms-usuarios:8081
      MS_RESERVAS_URL: http://ms-reservas:8082
      JWT_SECRET: clave_secreta_jwt_uteq_2026
    networks: [laboratorio-net]
networks:
  laboratorio-net:
    driver: bridge
volumes:
  pg-usuarios-data:
  pg-reservas-data:
  pg-asistencia-data:
  pg-inventario-data:
  pg-reportes-data:
```

XIV-D. Redes Docker y persistencia de datos

La red laboratorio-net es una red bridge definida por el usuario, que permite la comunicación entre contenedores mediante su nombre de servicio (p.ej., ms-usuarios puede ser resuelto por DNS interno a la dirección IP de ese contenedor). Los volúmenes con nombre (pg-usuarios-data, etc.) garantizan que los datos de PostgreSQL persistan, aunque los contenedores sean eliminados y recreados, lo cual es esencial para el ciclo de desarrollo iterativo.

XV. DIAGRAMA DE DESPLIEGUE

El diagrama de despliegue describe la distribución de los artefactos de software sobre la infraestructura de nodos de ejecución. En el entorno de desarrollo definido para este proyecto, todos los contenedores se ejecutan sobre un único host Windows 10/11 con Docker Desktop.



El frontend web se sirve desde un servidor estático en el puerto 3000. Todas las peticiones del navegador van al API Gateway en el puerto 8080, que es el único puerto del ecosistema expuesto al host. Los microservicios y las bases de datos únicamente son accesibles dentro de la red Docker interna laboratorio-net, garantizando aislamiento y seguridad.

XVI. ENTORNO DE DESARROLLO

La siguiente tabla especifica el entorno de desarrollo estándar del equipo ForáCode, garantizando uniformidad entre todos los integrantes mediante el uso de Docker.

Componente	Versión / Especificación	Uso en el proyecto
Sistema Operativo	Windows 10 / Windows 11 (64-bit)	Host de desarrollo y Docker Desktop
Java JDK	Eclipse Temurin 21 LTS	Compilación de todos los microservicios
Spring Boot	3.3.x (Spring Framework 6.x)	Framework base de todos los microservicios
Spring Cloud Gateway	2023.0.x	Implementación del API Gateway
PostgreSQL	16.x (imagen Docker alpine)	Motor de base de datos relacional
Docker Desktop	4.x (con WSL2 backend)	Contenerización y orquestación local
Maven	3.9.x (wrapper incluido)	Gestión de dependencias y build
IntelliJ IDEA	Community 2024.x o superior	IDE principal de desarrollo
Git / GitHub	Git 2.x / GitHub (repositorio público)	Control de versiones y colaboración
Postman	Última versión estable	Pruebas manuales del contrato API REST

JMeter	5.6.x	Pruebas de carga y acceso concurrente (Entrega 3)
--------	-------	---

XVII. PLAN DE IMPLEMENTACIÓN PARA LA SIGUIENTE ENTREGA

El plan de implementación cubre las seis semanas siguientes a la entrega del presente documento. Está orientado a alcanzar un sistema completamente funcional para la Entrega 3, donde se presentarán las evidencias experimentales que permitan responder las preguntas de investigación.

Sem.	Actividad principal	Resultado esperado
1	Configuración del entorno de desarrollo con Java 21, Spring Boot 3.x, Docker Desktop e IntelliJ IDEA. Creación del repositorio en GitHub con estructura multi-módulo Maven.	Entorno funcional verificado; estructura de proyectos en GitHub.
2	Implementación del ms-usuarios con autenticación JWT. Configuración del API Gateway (Spring Cloud Gateway). Pruebas de enrutamiento básico.	ms-usuarios operativo detrás del Gateway. Token JWT generado y validado.
3	Implementación de ms-reservas y ms-inventario. Integración con sus bases de datos PostgreSQL independientes. Pruebas de integración entre ambos servicios.	CRUD completo de reservas e inventario con persistencia separada.
4	Implementación de ms-asistencia y ms-reportes. Comunicación REST entre ms-asistencia y ms-reservas para validar reservas activas.	Flujo completo de registro de asistencia vinculado a reservas activas.
5	Contenerización completa: Dockerfile por microservicio, docker-compose global. Validación del entorno distribuido completo levantado con un único comando.	docker-compose up levanta todo el ecosistema de microservicios sin errores.
6	Pruebas de carga básicas con JMeter. Documentación de la API REST con Swagger/OpenAPI. Revisión de seguridad del API Gateway. Preparación de la Entrega 3.	Evidencia de acceso concurrente documentada. API REST completamente especificada.

XVIII. CONCLUSIONES PARCIALES

La presente entrega concreta el diseño arquitectónico del Sistema Distribuido para el Control y Administración de Laboratorios Informáticos, estableciendo las bases técnicas sobre las cuales se construirá la implementación en la Entrega 3.

En primer lugar, la transición de una arquitectura monolítica conceptual hacia una arquitectura de microservicios responde de forma directa a las preguntas de investigación del proyecto [16]. La separación funcional en cinco microservicios independientes establece contratos explícitos de comunicación mediante APIs REST [24], lo que constituye una respuesta técnica concreta a la primera pregunta de investigación sobre si la incorporación de principios distribuidos mejora la comunicación entre módulos.

En segundo lugar, el diseño del API Gateway como punto de entrada único al ecosistema de microservicios resuelve el problema del acoplamiento entre el frontend y los servicios internos [23]. Esta decisión, documentada en ADR-002, centraliza la autenticación, el enrutamiento y el monitoreo, simplificando el mantenimiento del sistema sin sacrificar la independencia de los microservicios.

En tercer lugar, la aplicación del patrón Database per Service, documentado en ADR-003, garantiza el aislamiento completo de la persistencia de datos [22]. Cada microservicio es el único propietario de su esquema de datos, eliminando el acoplamiento oculto que existe en sistemas monolíticos donde múltiples módulos acceden a las mismas tablas de base de datos.

En cuarto lugar, la estrategia de contenerización con Docker y docker-compose establece un entorno de desarrollo reproducible y aislado, eliminando la variabilidad de configuración entre los equipos del equipo de desarrollo [19]. Esta decisión sienta también las bases para una futura migración hacia plataformas de orquestación de contenedores como Kubernetes [25].

Finalmente, el contrato API REST especificado en esta entrega representa el artefacto de integración central del proyecto: define con precisión los mensajes JSON de entrada y salida de cada endpoint [21], lo que permitirá al equipo desarrollar los microservicios de forma paralela y relativamente independiente, validando la integración en las etapas finales de la Entrega 3. Las evidencias experimentales que respondan cuantitativamente las preguntas de investigación se presentarán en la siguiente entrega, una vez que la implementación esté completa.

XIX. REFERENCIAS

- [1] J. Yu, H. Chen, K. Wu, Z. Cai, and J. Cui, "A Distributed Storage System for Robust, Privacy-Preserving Surveillance Cameras," 2020 IEEE 40th Int. Conf. Distributed Computing Systems (ICDCS), vol. 2020-November, pp. 1195–1196, Nov. 2020, doi: 10.1109/ICDCS47774.2020.00189.
- [2] M. H. Syed et al., "Real-Time Coupling of Geographically Distributed Research Infrastructures: Taxonomy, Overview, and Real-World Smart Grid Applications," IEEE Trans. Smart Grid, vol. 12, no. 2, pp. 1747–1760, Mar. 2021, doi: 10.1109/TSG.2020.3033070.
- [3] D. Saleem, A. Sundararajan, A. Sanghvi, J. Rivera, A. I. Sarwat, and B. Kroposki, "A Multidimensional Holistic Framework for the Security of Distributed Energy and Control Systems," IEEE Syst. J., vol. 14, no. 1, pp. 17–27, Mar. 2020, doi: 10.1109/JSYST.2019.2919464.
- [4] J. Peng, B. Fan, Q. Yang, and W. Liu, "Distributed event-triggered control of dc microgrids," IEEE Syst. J., vol. 15, no. 2, pp. 2504–2514, Jun. 2021, doi: 10.1109/JSYST.2020.2994532.
- [5] C. Deng and G. H. Yang, "Distributed adaptive fault-tolerant control approach to cooperative output regulation for linear multi-agent systems," Automatica, vol. 103, pp. 62–68, May 2019, doi: 10.1016/j.AUTOMATICA.2019.01.013.
- [6] Y. R. Sturz, E. L. Zhu, U. Rosolia, K. H. Johansson, and F. Borrelli, "Distributed Learning Model Predictive Control for Linear Systems," Proc. IEEE Conf. Decision and Control, vol. 2020-December, pp. 4366–4373, Dec. 2020, doi: 10.1109/CDC42340.2020.9303820.
- [7] M. Casimiro, D. Didona, P. Romano, L. Rodrigues, W. Zwaenepoel, and D. Garlan, "Lynceus: Cost-efficient tuning and provisioning of data analytic jobs," Proc. Int. Conf. Distrib. Comput. Syst., vol. 2020-November, pp. 56–66, Nov. 2020, doi: 10.1109/ICDCS47774.2020.00047.
- [8] Z. Xu et al., "To cache or not to cache: Stable service caching in mobile edge-clouds of a service market," Proc. Int. Conf. Distrib. Comput. Syst., vol. 2020-November, pp. 421–431, Nov. 2020, doi: 10.1109/ICDCS47774.2020.00051.
- [9] S. Mousavi, D. Lee, T. Griffin, D. Steadman, and A. Mockus, "Collaborative Learning of Semi-Supervised Clustering and Classification for Labeling Uncurated Data," Proc. Int. Conf. Image Processing, ICIP, vol. 2020-October, pp. 1716–1720, Oct. 2020, doi: 10.1109/ICIP40778.2020.9191300.
- [10] N. Li, I. Kolmanovsky, A. Girard, and D. Filev, "Fuzzy Encoded Markov Chains: Overview, Observer Theory, and Applications," IEEE Trans. Syst. Man Cybern. Syst., vol. 51, no. 1, pp. 116–130, Jan. 2021, doi: 10.1109/TSMC.2020.3042960.
- [11] J. Wang et al., "Performance evaluation of distributed energy resource management via advanced hardware-in-the-loop simulation," 2020 IEEE Power and Energy Society Innovative Smart Grid Technologies Conf., ISGT 2020, Feb. 2020, doi: 10.1109/ISGT45199.2020.9087667.
- [12] I. L. D. Ridgley, R. A. Freeman, K. M. Lynch, "Private and Hot-Pluggable Distributed Averaging," IEEE Control Syst. Lett., vol. 4, no. 4, pp. 988–993, Oct. 2020, doi: 10.1109/LCSYS.2020.2996957.
- [13] M. Al-Saffar, M. Al-Saffar, and P. Musilek, "Reinforcement Learning-Based Distributed BESS Management for Mitigating Overvoltage Issues in Systems with High PV Penetration," IEEE Trans. Smart Grid, vol. 11, no. 4, pp. 2980–2994, Jul. 2020, doi: 10.1109/TSG.2020.2972208.
- [14] T. F. Stafford and H. Treiblmaier, "Characteristics of a Blockchain Ecosystem for Secure and Sharable Electronic Medical Records," IEEE Trans. Eng. Manag., vol. 67, no. 4, pp. 1340–1362, Nov. 2020, doi: 10.1109/TEM.2020.2973095.
- [15] J. M. Batalla, "On Analyzing Video Transmission over Wireless WiFi and 5G C-Band in Harsh IIoT Environments," IEEE Access, vol. 8, pp. 118534–118541, 2020, doi: 10.1109/ACCESS.2020.3005641.
- [16] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, "Microservices: Yesterday, Today, and Tomorrow," in Present and Ulterior Software Engineering, M. Mazzara and B. Meyer, Eds. Cham: Springer, 2017, pp. 195–216, doi: 10.1007/978-3-319-67425-4_12.
- [17] R. Heinrich, A. van Hoom, H. Knoche, F. Li, L. Lwakatare, C. Pahl, S. Schulte, and J. Wettinger, "Performance Engineering for

- Microservices: Research Challenges and Directions," in Proc. 8th ACM/SPEC Int. Conf. Performance Engineering Companion (ICPE Companion), pp. 223–226, Apr. 2017, doi: 10.1145/3053600.3053605.
- [18] P. Jamshidi, C. Pahl, N. C. Mendonca, J. Lewis, and S. Tilkov, "Microservices: The Journey So Far and Challenges Ahead," *IEEE Softw.*, vol. 35, no. 3, pp. 24–35, May/Jun. 2018, doi: 10.1109/MS.2018.2141039.
 - [19] C. Pahl, A. Brogi, J. Soldani, and P. Jamshidi, "Cloud Container Technologies: A State-of-the-Art Review," *IEEE Trans. Cloud Comput.*, vol. 7, no. 3, pp. 677–692, Jul./Sep. 2019, doi: 10.1109/TCC.2017.2702586.
 - [20] T. Cerny, M. J. Donahoo, and M. Trnka, "Contextual Understanding of Microservice Architecture: Current and Future Directions," *ACM SIGAPP Appl. Comput. Rev.*, vol. 17, no. 4, pp. 29–45, Dec. 2017, doi: 10.1145/3183628.3183631.
 - [21] G. Marquez, M. Osorio, and H. Astudillo, "Actual Use of Architectural Patterns in Microservices-Based Open Source Projects," in 2018 25th Asia-Pacific Software Engineering Conference (APSEC), pp. 31–40, Dec. 2018, doi: 10.1109/APSEC.2018.00016.
 - [22] L. Chen, "Microservices: Architecting for Continuous Delivery and DevOps," in 2018 IEEE International Conference on Software Architecture (ICSA), pp. 39–397, Apr. 2018, doi: 10.1109/ICSA.2018.00013.
 - [23] R. T. Fielding and R. N. Taylor, "Principled Design of the Modern Web Architecture," *ACM Trans. Internet Technol.*, vol. 2, no. 2, pp. 115–150, May 2002, doi: 10.1145/514183.514185.
 - [24] Z. Li, P. Liang, and A. Avgeriou, "Architectural Technical Debt in Microservices: An Empirical Study," in 2021 IEEE 18th Int. Conf. Software Architecture (ICSA), pp. 107–118, Mar. 2021, doi: 10.1109/ICSA51549.2021.00019.
 - [25] A. Balalaie, A. Heydarnoori, and P. Jamshidi, "Microservices Architecture Enables DevOps: Migration to a Cloud-Native Architecture," *IEEE Softw.*, vol. 33, no. 3, pp. 42–52, May/Jun. 2016, doi: 10.1109/MS.2016.64.

Repositorio GitHub del proyecto:

https://github.com/ffarinangog2/proyecto_final_aplicaciones_distribuidas-SCLI.git